1 Wprowadzenie

Pierwsze ćwiczenie, którym się zajmiemy, będzie związane z problemem sortowania danych. Dla ułatwienia dalszych rozważań przyjmijmy, że mówiąc o sortowaniu kolekcji liczb (lub łańcuchów znaków), będziemy rozważali porządek leksykograficzny.

Zarówno w życiu codziennym jak i w praktyce inżynierskiej dany problem może mieć wiele rozwiązań. Nie inaczej jest z zadaniem sortowania, dla którego powstało wiele różnych algorytmów. Wybór odpowiedniego algorytmu może być podyktowany szeregiem czynników, do których można zaliczyć m.in.: łatwość samodzielnej implementacji, złożoność obliczeniową (czasową, pamięciową), dostępność zasobów. W przypadku rozwiązywania problemów nieszablonowych, dla których nie istnieje gotowa instrukcja postępowania, poszukiwanie dobrego rozwiązania jest kompromisem pomiędzy satysfakcją z uzyskanego wyniku (złożonością algorytmu), a czasem poświęconym na jego implementację. Umiejętność świadomego wyważenia tych proporcji jest kompetencją inżynierską poszukiwaną na rynku pracy.

1.1 Złożoność algorytmów

Ocena czasu pracy algorytmu jak i zużywanych przez niego zasobów pamięciowych może być wykonana bardzo dokładnie np. poprzez zliczenie wszystkich instrukcji, które zostaną wykonane w trakcie jego działania.

Częściej jednak mówimy o klasach złożoności algorytmów przybliżając ich precyzyjnie wyliczone wartości asymptotycznymi złożonościami, co wynika z faktu, że najczęściej precyzyjnie wyliczone wartości są funkcjami rozmiaru danych n (np.: $f(n) = an^2+bn+c$, gdzie $a > 0$), a przy bardzo dużym rozmiarze danych n przyjmuje się, że składniki niższych rzędów i stały współczynnik przy składniku o najwyższym rzędzie stają się nieistotne (są pomijane w rozważaniach).

Do najpopularniejszych notacji zalicza się:

- notację O – do określenia asymptotycznego ograniczenia górnego;
- notację Ω – do określenia asymptotycznego ograniczenia dolnego;
- notację Θ – do określenia asymptotycznego dokładnego oszacowania.

Na podstawie tych rozważań można napisać, że funkcja $f(n)$ ($f(n) = an^2 + bn + c$, gdzie $a > 0$), określająca pesymistyczny czas działania algorytmu dla danych o rozmiarze n, jest $O(n^2)$ ($f(n) = O(n^2)$).

Biorąc to wszystko pod uwagę, zastanówmy się, jaka może być złożoność wyrażona w notacji O dla poniższych fragmentów kodu (1, 2), przy założeniu, że $numsA$ i $numsB$ składają się odpowiednio z a i b elementów.

Listing 1: Złożoność $O(a+b)$.

```
for(int a: numsA){
    .println(a);
}

for(int b: numsB){
    .println(b);
}
```

Listing 2: Złożoność $O(a*b)$.

```
for(int a: numsA){
    for(int b: numsB){
        .println(a + b);
    }
}
```

Ma to sens, prawda? To spróbujmy jeszcze dwa razy (listingi: 3, 4).

W przypadku listingu 3 wykonujemy dwie niezależne pętle, a w każdej z nich kręcimy się n razy, dlatego złożoność z $O(2n)$ redukujemy (pomijając stałe) do $O(n)$.

Listing 3: Złożoność $O(n)$.

```
void countProductAndSum(int[] nums){
    int product = 1;
    int sum = 0;
    int n = nums.length;

    for(int i = 0; i < n; i++){
        product *= nums[i];
    }

    for(int i = 0; i < n; i++){
        sum += nums[i];
    }

    System.out.println(
        format("Product: %d | Sum: %d", product, sum));
}
```

Ostatni przykład (listing 4) pokazuje nam dwie zagnieżdżone pętle – dla każdego kroku (n kroków) z pętli zewnętrznej wykonujemy n-krokową pętlę wewnętrzną, więc uzyskujemy złożoność $O(n^2)$.

Listing 4: Złożoność $O(n^2)$.

```
void printEachWithEach(int[] nums){
    int n = nums.length;

    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            System.out.println(
                format("%d, %d", nums[i], nums[j]));
```

```
            }
        }
}
```

1.2 Algorytmy sortowania

Praca na uporządkowanej (posortowanej) kolekcji elementów może być celem samym w sobie, ale może być również pewnym etapem przejściowym prowadzącym do celu. Niezależnie od powodu, dla którego chcemy otrzymać uporządkowaną kolekcję, warto jest znać różnice (zalety i wady) stosowania różnych algorytmów służących do tego celu.

Algorytmy różnią się pod względem m.in.: złożoności obliczeniowej (czasowej, pamięciowej), trudności implementacyjnej (gdy nie korzystamy z bibliotek gotowych implementacji), stabilnością sortowania, pracą w miejscu (łac. *in-situ*), szczególnym zachowaniem w przypadku określonego układu lub rozmiaru danych wejściowych.

Przykładowo, algorytm sortowania przez wybór (ang. *selection sort*) ma praktycznie taką samą złożoność czasową $O(n^2)$ niezależnie od układu danych wejściowych. A algorytm sortowania przez wstawianie (ang. *insertion sort*) dla małych rozmiarów danych wejściowych lub danych prawie uporządkowanych (wstępnie posortowanych) działa znacznie szybciej (nawet $\Theta(n)$).

Omówienie zarówno podstawowych jak i bardziej specyficznych algorytmów sortowania można znaleźć we *Wprowadzeniu do algorytmów* Cormena i in., dlatego te treści nie będą tutaj szczegółowo powielane. Warto jednak podkreślić kilka szczególnych faktów, które będą istotne przy wykonaniu pierwszego ćwiczenia laboratoryjnego.

Algorytm sortowania przez kopcowanie (ang. *heap sort*) swoje działanie opiera na właściwościach tablicowej struktury danych jaką jest **kopiec binarny** – wygodnie jest ją rozpatrywać teoretycznie jako drzewo binarne. Najczęściej mówimy o **maksymalnym kopcu**, co oznacza, że w jego korzeniu znajduje się największa wartość, a relacja między rodzicem a potomkami (czyli **własność kopca**) określa, że rodzic nie może być mniejszy od swoich dzieci. Kopiec ten jest również **kopcem zupełnym** (drzewem), a tym samym liście w tym kopcu mogą występować tylko na dwóch ostatnich poziomach. Spełnienie **własności kształtu** oznacza, że nowi potomkowie dołączają do drzewa na pierwszą wolną pozycję.

Algorytm szybkiego sortowania (ang. *quick sort*) jest jednym z częściej wykorzystywanych algorytmów sortujących (wraz ze swoimi ulepszeniami i wariantami implementacyjnymi). Nie jest on jednak algorytmem, który jest najlepszym wyborem do każdego zadania, choć zdecydowanie korzystnie wypada w użyciu dla typowych, „codziennych" sytuacji.

Algorytm ten występuje w dwóch popularnych wariantach zaproponowanych przez Sir Tony'ego Hoare'a i Nico Lomuto. Jest to algorytm z grupy „dziel i zwyciężaj", co oznacza (jak wynika z nazwy), że składa się z ~~dwóch~~ trzech etapów:

dziel, zwyciężaj, połącz. A przytoczone warianty różnią się właśnie wykorzystaną metodą podziału.

Najwygodniej jest (tylko dla celów poglądowych, ale **nigdy dla docelowej implementacji**) przedstawić algorytm szybkiego sortowania przy pomocy rekurencji. Nie jest to jednak zalecany sposób wykonywania implementacji ze względu na koszt (i ryzyko błędu) działania takiego kodu.

Listing 5: QuickSort (met. Lomuto) – poglądowa wersja rekurencyjna

```
void quickSort(double[] nums, int left, int right) {
    if (left < right) {

        int pivotId = partition(nums, left, right);

        quickSort(nums, left, pivotId - 1);
        quickSort(nums, pivotId + 1, right);
    }
}
```

Listing 6: QuickSort (met. Hoare'a) – poglądowa wersja rekurencyjna

```
void quickSort(double[] nums, int left, int right) {
    if (left < right) {

        int pivotId = partition(nums, left, right);

        quickSort(nums, left, pivotId);
        quickSort(nums, pivotId + 1, right);
    }
}
```

Listingi 5. i 6. pokazują przykładową rekurencyjną implementację algorytmu szybkiego sortowania przy założeniu, że metoda `partition(...)` wykonuje podział odpowiednio podejściem Lomuto lub Hoare'a.

Z pokazanego kodu wyraźnie widać (przy założeniu, że ten kod jest prawidłowy), że najistotniejszym elementem w algorytmie szybkiego sortowania jest właśnie procedura podziału zwracająca indeks elementu osiowego (rozdzielającego). Celem wykonania podziału jest przeniesienie elementów w taki sposób, żeby po lewej stronie były elementy mniejsze (bądź równe) wartości rozdzielającej, a po prawej stronie – elementy większe.

Różnica w podejściu Hoare'a i Lomuto związana jest z tym, który element zostaje wybrany jako element rozdzielający, i w którym miejscu (po podziale) ostatecznie znajduje się ten element.

Algorytmem sortującym szczególnego przeznaczenia (nie dla każdych danych) jest algorytm sortowania przez zliczanie (ang. *counting sort*). Ten algorytm dla zestawu danych (powtarzających się liczb z góry ograniczonego zakresu znacznie mniejszego niż rozmiar danych) może działać ze złożonością $O(n)$. Trzeba jednak ciągle mieć na uwadze, że nie jest to algorytm ogólnego przeznaczenia.

A najistotniejszym novum – względem pozostałych algorytmów – jest wykorzystanie dodatkowej pamięci na stworzenie `nOfCounters` liczników zliczających wystąpienie poszczególnych wartości przypisanych do danego kubełka.

Listing 7: Counting sort

```
void countingSort(int[] nums, int maxVal) {
    // ... validateInput

    int n = nums.length;
    int nOfCounters = maxVal + 1;

    int[] counters = new int[nOfCounters];

    for (int i = 0; i < n; i++) {
        counters[nums[i]]++;
    }

    int j = 0;
    for (int i = 0; i < nOfCounters; i++) {
        while (counters[i] > 0) {
            nums[j++] = i;
            counters[i]--;
        }
    }
}
```

2 Zadania

2.1 Zadanie 1.

Proszę, wykorzystując notację wielkiego O, określić asymptotyczną złożoność czasową trzech poniższych fragmentów kodu.

Lp.	Nr przykładu	Złożoność O(?)
1.	A	O(n^2)
2.	B	O(numsA.length * numsB.length)
3.	C	O(n)

Listing 8: Zad. 1 (A)

```
void printHalfOfPairs(int[] nums) {
    int n = nums.length; // may be null!

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
```

```
                System.out.println(
                    format("(%d, %d)", nums[i], nums[j]));
            }
        }
    }
```

Listing 9: Zad. 1 (B)

```
void printSmallerPairs(int[] numsA, int[] numsB) {
    // may be nulls!
    for (int i = 0; i < numsA.length; i++) {
        for (int j = 0; j < numsB.length; j++) {
            if (numsA[i] < numsB[j]) {
                System.out.println(
                  format("(%d, %d)", numsA[i], numsB[j]));
            }
        }
    }
}
```

Listing 10: Zad. 1 (C)

```
void reverse(int[] nums) {
    int n = nums.length; // may be null!
    int rightId, leftVal;

    for (int i = 0; i < n / 2; i++) {
        rightId = n - i - 1;
        leftVal = nums[i];
        nums[i] = nums[rightId];
        nums[rightId] = leftVal;
    }
}
```

2.2 Zadanie 2.

Proszę określić (**T**ak lub **N**ie), który z poniższych zapisów jest tożsamy asymptotycznej złożoności $O(n)$ (wiadomo, że dla dużej wartości n). Proszę podać krótkie uzasadnienie (najlepiej jako równoważnik zdania).

Lp.	Przykład	Czy jest O(n)?	Uzasadnienie
1.	O(n+k), gdzie k <n/3	Tak	O(n+(1/3)n) = O((4/3)n) = O(n)
2.	O(2n)	Tak	Stała ominięta
3.	O(n+m)	Nie	n i m to nie stałe
4.	O(n+log(n))	Tak	n rośnie szybciej niż log(n)

2.3 Zadanie 3.

W kodzie źródłowym zostały umieszczone dwie klasy (`QuickSortIterativeHoare`, `QuickSortIterativeLomuto`) dla implementacji algorytmu szybkiego sortowania z wykorzystaniem metod podziału Hoare'a i Lomuto. Proszę wybrać jedną z tych metod podziału, a następnie proszę zaimplementować trzy klasy teoretycznie usprawniające działanie algorytmu szybkiego sortowania:

- `QuickSortIterativeMedian3` – wybór elementu rozdzielającego (pivot) jako mediany z trzech wartości;
- `QuickSortIterativeRandom` – wybór elementu rozdzielającego (pivot) jako elementu losowo wybranego;
- `QuickSortIterativeWithInSort` – wykorzystanie sortowania przez wstawianie dla małych ($n < 20$) podproblemów.

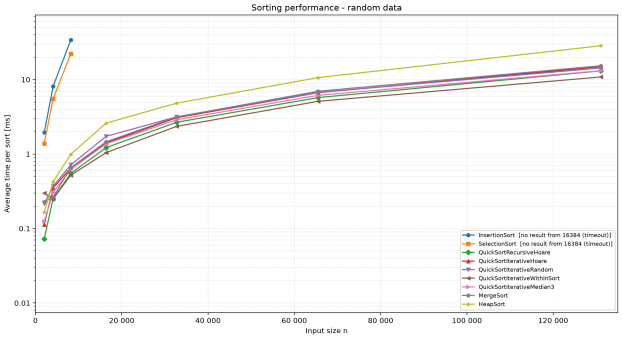
2.4 Zadanie 4.

Proszę, wykorzystując otrzymany kod uzupełniony o własne implementacje, wykonać wykresy (nie, nie trzeba pisać kodu tworzącego wykres – arkusz kalkulacyjny jest wystarczająco dobrym narzędziem) pokazujący, jak zmienia się czas wykonania algorytmu w zależności od rozmiaru danych wejściowych. Proszę wykonać wykresy w skali logarytmicznej na osi OY. Na podstawie otrzymanych wykresów, proszę uzupełnić poniższą tabelę.

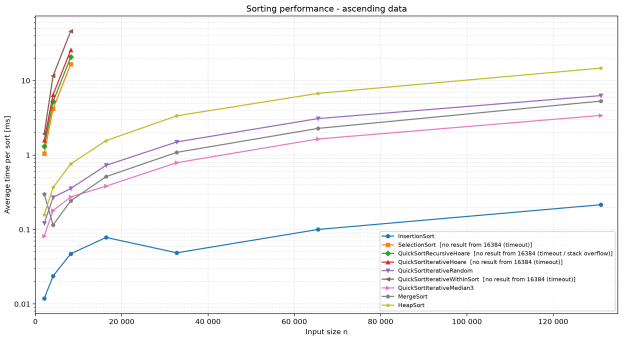
Do tego celu można wykorzystać pakiet testów `performance.table`. Zadanie dla chętnych: zamiast wspomnianego pakietu, proszę wykorzystać pakiet `performance.charts`. Trzeba w nim wprowadzić trochę zmian, aby nie było konieczności tworzenia kilkudziesięciu klas, ale dzięki temu wykres będzie bardziej „estetyczny" (dokładniejszy).

W pomiarach, dla których algorytm przekroczył limit czasu, wynik oznaczony jako `-1` pominięto. Ocena najlepszego i najgorszego algorytmu została wykonana na podstawie poprawnych pomiarów dla największych rozmiarów danych, dla których porównywane algorytmy zakończyły działanie.

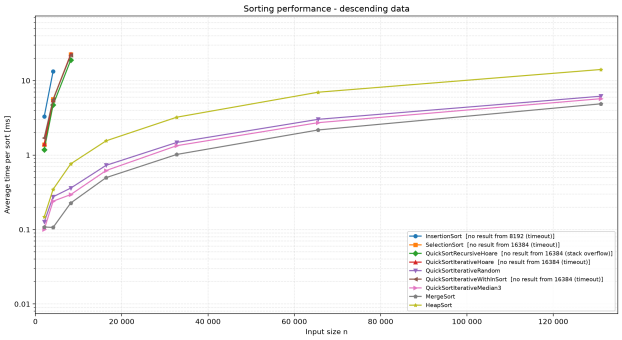
Lp.	Kategoria	Algorytm (lub algorytmy)
1.	Najlepszy algorytm dla danych losowych	QuickSortIterativeWithInSort
2.	Najlepszy algorytm dla danych rosnących	InsertionSort
3.	Najlepszy algorytm dla danych malejących	MergeSort
4.	Najgorszy algorytm dla danych losowych	InsertionSort
5.	Najgorszy algorytm dla danych rosnących	SelectionSort
6.	Najgorszy algorytm dla danych malejących	SelectionSort



(a) Czas sortowania dla danych losowych.



(b) Czas sortowania dla danych rosnących.



(c) Czas sortowania dla danych malejących.

Rysunek 1: Algorytmy: InsertionSort, SelectionSort, QuickSortRecursiveHoare, QuickSortIterativeHoare, QuickSortIterativeRandom, QuickSortIterativeWithInSort, QuickSortIterativeMedian3, MergeSort oraz HeapSort.

2.5 Zadanie 5.

Proszę, wykorzystując otrzymany kod uzupełniony (i poprawiony) o własne implementacje, uzupełnić klasę testów jednostkowych `MergeSortStabilityTest` o

metody pokazujące, czy sortowanie przez scalanie jest algorytmem sortującym stabilnie. Przykładowy (wybranych z samodzielnie zaimplementowanych) test proszę umieścić w listingu 11.

Listing 11: Przykładowa metoda testowa dla MergeSort pokazująca, że algorytm jest stabilny.

```
// TODO: placeholder for test method
```

2.6 Zadanie 6.

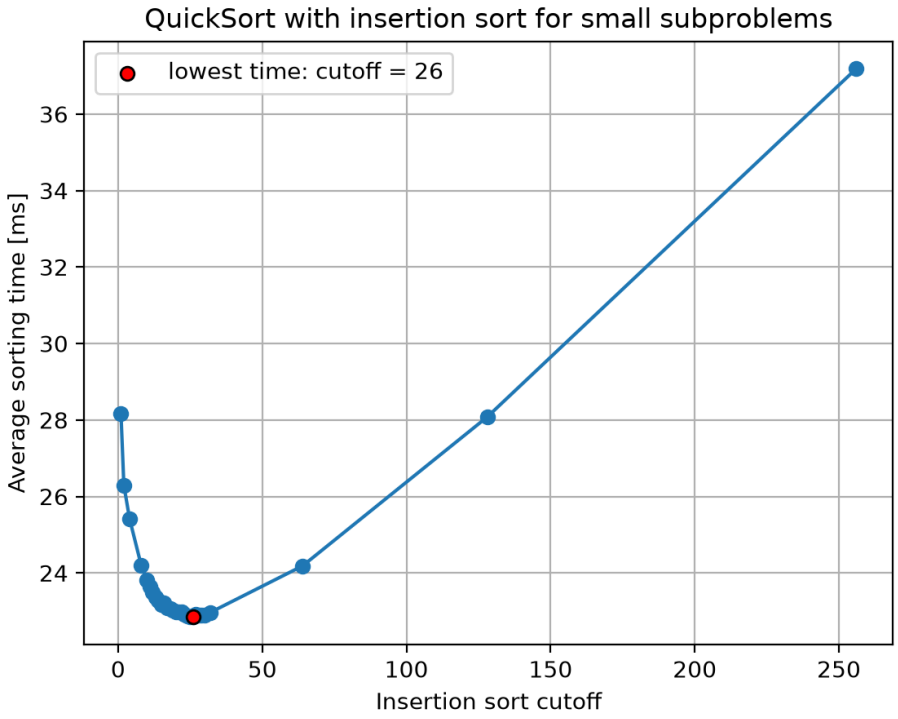
Proszę, wykorzystując otrzymany kod uzupełniony o własne implementacje, uzupełnić poniższą tabelę pokazującą, jaki wpływ na czas sortowania ma typ (prosty, złożony) sortowanych danych. W wybranych klasach konieczne jest zaimplementowanie dodatkowego interfejsu `Sorting` (oraz dodatkowych klas testów). Proszę krótko skomentować otrzymane wartości.

Lp.	Rozmiar	InsertionSort		ReferenceSort		SelectionSort	
		Integer	int	Integer	int	Integer	int
1.	1024						
2.	2048						
3.	4096						
4.	8192						
5.	16384						
6.	32768						
7.	65536						
8.	131072						
9.	262144						
10.	524288						

Komentarz (a.k.a. wnioski): **TODO** ——————————————————————.

2.7 Zadanie 7.

Proszę, wykorzystując otrzymany kod, ustalić, jaka jest optymalna wartość podproblemów podczas wykonywania algorytmu szybkiego sortowania na załączenie sortowania przez wstawianie. Przeprowadzoną analizę proszę zobrazować na wykresie.



Rysunek 2: Wykres pokazuje, jak zmienia się czas sortowania tablicy losowo uporządkowanych wartości algorytmem szybkiego sortowania (z dołączonym algorytmem sortowania przez wstawianie), w zależności od wielkości podproblemu, dla którego dołączany jest ten algorytm (sortowania przez wstawianie).

Komentarz (a.k.a. wnioski): Optymalna wartość rozmiaru podproblemów wynosi coś około 20-30 dla mojego środowiska. Za każdym razem gdy odpalałem testy to wartości otrzymywałem w tym zakresie. Wartość ta może się różnić w zależności od środowiska, w którym wykonywane są testy.

2.8 Zadanie 8.

Proszę samodzielnie przygotować pakiet testów wydajnościowych dla algorytmu sortowania przez zliczanie. Testy powinny pomóc w przygotowaniu wykresu pokazującego, jak zmienia się czas sortowania danych, gdy zestaw danych jest zestawem „przypadkowym” (czyli bez powtórzeń), oraz w przypadku, gdy zestaw danych jest zestawem preferowanym przez algorytm sortowania przez zliczanie (z wieloma powtórzeniami).

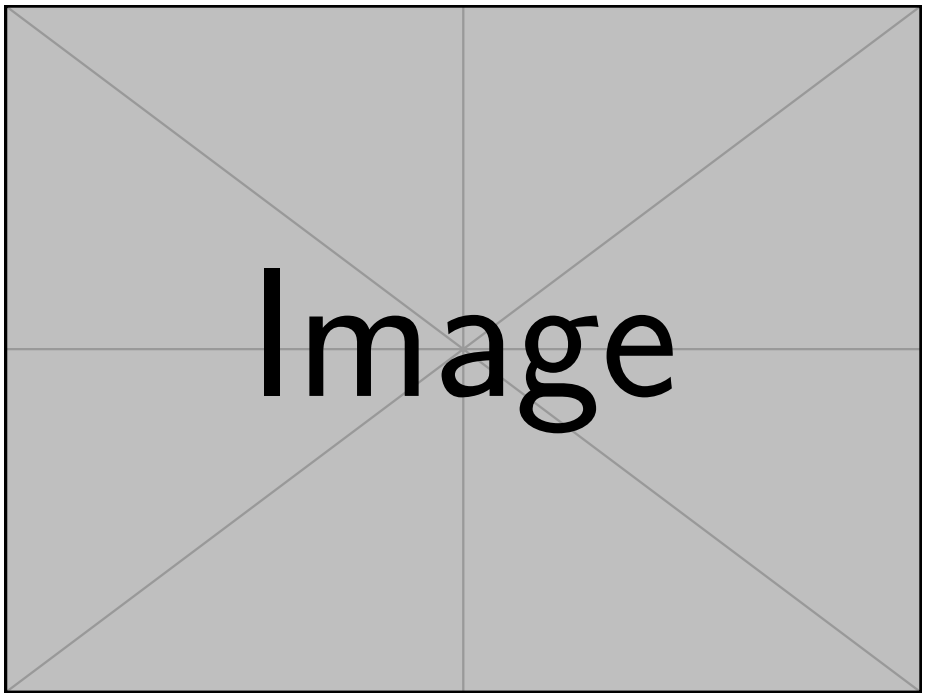
Miejsce na wykres oraz komentarz również należy stworzyć sobie samodzielnie w tej sekcji.

2.9 Zadanie 9.

Proszę, korzystając z kodu źródłowego Javy oraz dokumentacji (Javadoc), podać, kto jest autorem wykorzystywanego algorytmu sortowania dla tablicy liczb całkowitych (`int[]`). Proszę również określić, czym szczególnym charakteryzuje się ta implementacja.

Proszę na dowód załączyć zrzut ekranu pokazujący istotny fragment dokumentacji (Javadoc).

Odpowiedź: **TODO** ——————————————————————————.



Rysunek 3: Zrzut ekranu z fragmentem dokumentacji dla metody sortującej tablicę liczb całkowitych. Pokazana jest nazwa klasy wraz z opisem, kto jest jej autorem.

2.10 Zadanie 10.

Proszę zweryfikować poprawność otrzymanych i wykonanych implementacji algorytmów poprzez napisanie testów jednostkowych. Każda klasa implementująca algorytm powinna być przetestowana.

Proszę to zadanie wykonać w sposób przemyślany (kopiowanie i powielanie kodu testów pomiędzy klasami jest ostatnim krokiem, na który należy się zdecydować). W celu zniwelowania redundancji kodu testów, proszę wykorzystać cechy paradygmatu programowania obiektowego.